

Deep Learning — Spring 2026

Assignment 5: From Supervised to Self-Supervised Learning

A Guided Implementation of SimCLR

Muhammad Abdul Aleem

Roll No: MSDS25022

Information Technology University, Lahore

Random Seed: 2026

GitHub Repository:

https://github.com/Abdul-Aleem-git/DL_Assignment5_SimCLR_MSDS25022

Model Weights (Google Drive):

<https://drive.google.com/drive/folders/1AiyurwKlz5yRbe3dO8XLKjN1CJJlhSd8?usp=sharing>

Task 0: Conceptual Warm-up

Q1. What does a supervised model learn from?

In supervised classification the model learns from image-label pairs. At every training step the cross-entropy loss penalises the network whenever its predicted class probabilities disagree with the ground-truth label. Over many epochs the network learns to extract features that are discriminative enough to separate the ten CIFAR-10 categories. Without labels there is no loss signal and, under standard supervised training, no learning.

Q2. Do unlabeled CIFAR-10 images still contain useful visual structure?

Yes. Labels are human-assigned annotations; they do not change the pixel content of an image. An unlabeled photograph of a dog still contains edges, textures, colour distributions, and object parts that distinguish it from, say, an airplane. The visual structure is there regardless of whether a label string is attached. Self-supervised methods like SimCLR exploit this structure by defining a pretext task directly from the image data itself, with no need for manual annotations.

Q3. Should two augmented views of the same image have similar representations?

Yes. Two randomly cropped, colour-jittered, or grayscaled versions of the same photograph still depict the same object. Their semantic content is identical; only irrelevant surface-level variations differ. A well-trained encoder should therefore map both views to nearby points in feature space. SimCLR enforces this explicitly by treating the pair as a positive example and maximising their cosine similarity via the NT-Xent loss.

Q4. Should views from different images have similar representations?

In general, no. Two images from different original photographs are likely to depict different objects or scenes. The contrastive objective pushes their representations apart so the encoder learns features that are specific to each image's content rather than collapsing everything to a single point. This repulsion is what prevents trivial solutions where the encoder ignores input entirely.

Q5. Why can a label-free model still be useful for classification?

Even without labels, a model trained on a well-designed pretext task must learn to distinguish images from one another in feature space. To push representations of two augmented views of dog images together and away from airplane images, the encoder implicitly learns that dogs and airplanes look different. These learned features capture semantically meaningful structure. A simple linear classifier trained on top of frozen features can then separate classes using only a small amount of labeled data, because the hard perceptual work has already been done during pretraining.

Q6. What is the difference between pretraining and fine-tuning?

Pretraining is the initial training phase in which the encoder learns general-purpose representations, typically from large amounts of unlabeled data using a self-supervised objective such as contrastive learning. The model does not learn to classify; it learns to organise feature space. Fine-tuning is a subsequent phase in which the pretrained weights are used as the starting point and the entire network is updated end-to-end on a downstream task (here, CIFAR-10 classification) using labeled data. Because fine-tuning starts from a sensible initialisation rather than random weights, it converges faster and generalises better, especially when the labeled dataset is small.

Task 1: Supervised Baseline with Limited Labels

A ResNet-18 was trained from scratch using only the fixed 10 % labeled split (5 000 images, 500 per class). The model used cross-entropy loss, Adam optimiser with $lr = 3e-4$, and a cosine learning rate schedule over 30 epochs. Augmentations were random crop (padding 4) and random horizontal flip.

Model	Test Accuracy
Supervised ResNet-18 (10% labels)	70.41%

The loss curves show the model converging steadily. Training loss dropped from 1.87 at epoch 1 to 0.15 by epoch 30, while validation loss reached its minimum of 0.94 at epoch 18, after which the model was lightly overfitting. The best checkpoint at epoch 18 was used for the final test evaluation.

The confusion matrix showed the model struggling most with cat/dog and automobile/truck pairs — both are visually similar categories that challenge any model trained with limited data.

Task 2: Understanding Augmentations

The SimCLR augmentation pipeline consists of five stages applied stochastically:

- RandomResizedCrop (scale 0.2–1.0) — forces the model to be invariant to scale and position
- RandomHorizontalFlip (p=0.5) — left-right orientation invariance
- ColorJitter (brightness, contrast, saturation 0.4; hue 0.1) — colour invariance
- RandomGrayscale (p=0.2) — partial colour invariance
- Normalise with CIFAR-10 mean and std

The TwoViewTransform wrapper applies this pipeline twice independently to the same PIL image, producing two views that share semantic content but differ in appearance. Ten example triplets (original, View A, View B) are shown in results/augmentation_examples.png.

Answers to augmentation questions:

- The two views are NOT identical — each application samples different random parameters.
- Both views still clearly represent the same object; the semantic content is preserved.
- SimCLR should treat them as a positive pair precisely because they share the same underlying identity.
- Too-weak augmentations: the two views look almost the same, making the task trivial. The model can match views using low-level pixel statistics rather than semantic features.
- Too-strong augmentations: the two views may lose enough shared information that even a human could not tell they came from the same image. The loss becomes impossible to minimise and training diverges.

Task 3: Feature Similarity Before Training

Before any SimCLR training, a randomly initialised ResNet-18 encoder was used to extract 512-dimensional features for a batch of 16 images (producing 32 views total). The cosine similarity matrix was computed and visualised.

Pair Type	Average Cosine Similarity
Same image, two augmented views	0.9910
Different images	0.9858

With a random encoder, positive pairs (same image) and negative pairs (different images) have almost identical average similarities. The encoder has no notion of which images are related — it maps all inputs to a roughly uniform distribution in feature space. This confirms that the SimCLR training objective is needed to create meaningful structure.

Task 4: SimCLR Implementation

4.1 Architecture

The encoder is ResNet-18 modified for 32x32 CIFAR images: conv1 replaced with a 3x3 convolution (stride 1, padding 1), and maxpool replaced with Identity. This preserves spatial resolution in early layers. The encoder outputs a 512-dimensional feature vector from the average pooling layer.

The projection head maps 512 $\rightarrow$ 256 (Linear + ReLU) $\rightarrow$ 128 (Linear). It is used only during SimCLR pretraining and discarded before downstream evaluation.

4.2 Positive and Negative Pairs

For a batch of N original images, TwoViewTransform produces $2N$ augmented views. Views i and $i+N$ form the positive pair for image i . All other $2N-2$ pairs involving anchor i are treated as negatives.

Original Image	View-A Index	View-B Index	Positive Pair
image 0	0	4	yes
image 1	1	5	yes
image 2	2	6	yes
image 3	3	7	yes

4.3 Similarity Matrix

The diagonal is ignored because $\text{sim}(z_i, z_i) = 1$ always; including it would make the denominator of the NT-Xent loss trivially dominated by self-similarity and distort gradients. Positive pairs sit at off-diagonal positions $[i, i+N]$ and $[i+N, i]$. All other pairs are treated as negatives under the closed-world assumption that any two different original images are semantically dissimilar.

4.4 NT-Xent Loss

The NT-Xent loss was implemented from scratch without any library. For each anchor view i , the loss is the negative log-probability of its positive partner under the row-wise softmax over all $2N-1$ non-self entries, scaled by temperature $\tau = 0.5$. Smaller τ sharpens the distribution, making the task harder and forcing the encoder to produce more discriminative representations.

Sanity check: with random inputs, the loss converges to approximately $\log(2N-1) = \log(15) = 2.71$. Our implementation produced 2.64, confirming correctness.

Task 5: SimCLR Pretraining

Training Settings

Setting	Value Used
Batch size	64
SimCLR epochs	50
Linear probe epochs	20
Fine-tuning epochs	20
Learning rate	3e-4
Optimizer	Adam
Temperature tau	0.5
GPU/CPU	CPU
Approx training time	~4 hours

Results

Pair Type	Before SimCLR	After SimCLR
Same image, two views	0.9910	0.8875
Different images	0.9858	-0.0222

The results confirm that SimCLR training worked correctly. Before training, positive and negative pair similarities were nearly identical (0.991 vs 0.986), meaning the random encoder placed all representations in a similar region of feature space with no structure. After 50 epochs, positive-pair similarity dropped to 0.888 while negative-pair similarity collapsed to -0.022 — the encoder learned to bring augmented views of the same image close together and push representations of different images apart. The NT-Xent loss decreased steadily from 3.87 at epoch 1 to 3.22 at epoch 50, indicating stable convergence.

Task 6: Linear Probe Evaluation

The encoder was frozen and a single linear layer (512 -> 10) was trained on the 10% labeled split for 20 epochs. This isolates the quality of the learned representations — if the frozen features are well-organised, even a linear classifier should perform well.

Model	Encoder	Test Accuracy
Random Linear Probe	Random frozen	24.69%
SimCLR Linear Probe	SimCLR frozen	72.10%

The random encoder achieves only 24.69% — marginally above chance (10%) — confirming that random features contain almost no useful class information. The SimCLR encoder achieves 72.10% with the same linear head, a 47-point improvement purely from the quality of pretrained features. This demonstrates that SimCLR learned genuinely useful visual representations without ever seeing a single label during pretraining.

Task 7: Fine-tuning the SimCLR Encoder

The SimCLR pretrained encoder was fine-tuned end-to-end with a classification head on the 10% labeled split for 20 epochs. Unlike linear probing, all encoder weights were updated, allowing the features to adapt specifically to CIFAR-10 classification.

Final Comparison Table

Model	Labels in Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18, 10% labels	Yes	No	70.41%
Random encoder + linear probe	No	Yes	24.69%
SimCLR encoder + linear probe	No	Yes	72.10%
SimCLR encoder + fine-tuning	No (pretraining) / Yes (ft)	No	83.35%

Fine-tuning the SimCLR encoder achieves 83.35%, which is 13 percentage points above the supervised baseline trained with the same 10% labels. This is the key result of the assignment: by pretraining on 40 000 unlabeled images, SimCLR learns representations that are far superior to those learned from 5 000 labeled images alone. When fine-tuning is then applied on the same 5 000 labeled images starting from those representations, the improvement is substantial.

Task 8: PCA / t-SNE Feature Visualisation

t-SNE (perplexity=40, max_iter=1000, seed=2026) was applied to 512-dimensional features extracted from 1000 fixed validation images using three encoder states.

- Random encoder: Points are scattered with no visible class grouping. The 10 classes overlap almost completely, confirming that random features carry no semantic information.
- SimCLR encoder: Clear loose clusters begin to appear. Several classes (airplane, automobile, ship) form reasonably tight groups. Animal categories (cat, dog, deer, horse) remain partially mixed, as these classes share textures and animal-like features that SimCLR cannot fully separate without label guidance.
- Fine-tuned encoder: The tightest and most separated clusters. Most classes form distinct regions in 2D space with minimal overlap. The improvement over SimCLR-only is visible, confirming that supervised fine-tuning refines the representations for the specific classification task.

Confused class pairs: cat/dog and automobile/truck remain the hardest to separate across all three encoders, consistent with their visual similarity.

Discussion: What Did SimCLR Learn?

SimCLR learned to produce representations that capture the semantic identity of an image independent of its superficial appearance. The core insight is that two randomly augmented views of the same image should look similar in feature space because they depict the same object, while views from different images should look different.

The similarity analysis is the clearest evidence of this. With random weights, positive and negative pairs were indistinguishable in feature space (0.991 vs 0.986). After SimCLR pretraining, positive pairs still had high similarity (0.888) while negative pairs dropped sharply to near-zero (-0.022). The encoder learned an internal representation where semantic identity is encoded in the direction of the feature vector, not its magnitude.

The linear probe result (72.10%) is perhaps the most interesting finding. A single matrix multiplication on top of frozen SimCLR features nearly matches the supervised baseline (70.41%) that used explicit labels during training. This means SimCLR organised feature space in a way that is largely linearly separable by class — without ever being told what classes exist. Fine-tuning then pushes this to 83.35% by allowing the full network to specialise further for classification.

Implementation Notes and Limitations

- Training was performed on CPU due to absence of a CUDA-capable GPU. SimCLR pretraining took approximately 4 hours for 50 epochs.
- The batch size was 64 (minimum allowed). Larger batches (256-512) typically improve SimCLR quality because more negative pairs are available per update.
- The loss decreased from 3.87 to 3.22 over 50 epochs, which is steady but the curve had not fully plateaued, suggesting more epochs would further improve representations.
- metrics.json shows zeros for random/SimCLR probe similarity fields because linear_probe_results.json was not separately generated. The actual values are reported in the tables above.

Appendix: Training Settings Used

Item	Value Used
Batch size	64
SimCLR pretraining epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	3e-4
Optimizer	Adam
Temperature tau	0.5
Projection dimension	128
Random seed	2026
GPU/CPU	CPU
Approx SimCLR training time	~4 hours